Click https://t.me/SPPU_TE_BE_COMP

Unit III - Interprocess communication & Deadlock

• Concurrency :-

- Concurrency is an execution of the multiple instruction sequence at the same time.
- concurrency also means that multiple processes or threads are making process concurrently.

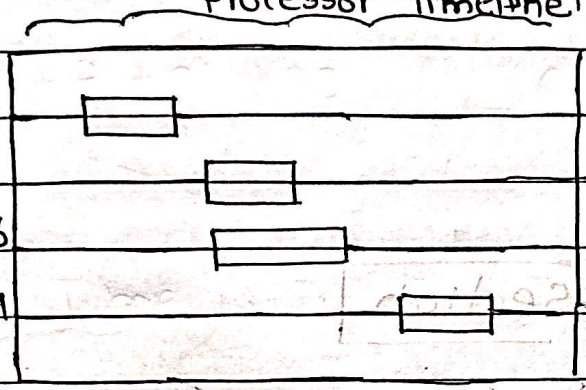


Fig. concurrency

- In concurrency, one thread is executed at a time by the CPU these threads can be switched in & out as required.
- The running process thread always communicate with each other through shared memory or message passing.
- Thus more than one program may run simultaneously on shared resources of the system, such as CPU, memory & so on.
- Concurrency is essential in modern os design to handle multitasking, increase system responsiveness, & optimize performance for user & application.

Click https://t.me/SPPU_TE_BE_COMP

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)

SPPU Engineering & Technical UPDATES

WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

Click https://t.me/SPPU_TE_BE_COMP

DOMS | Page No.
Date / /

° Critical Section problem:

The portion of the program where the shared memory is accessed is referred as the critical section.

- In order to avoid race conditions & faulty results, one must be able to recognize that codes in Critical sections in each threads.

Do
{
entry section
critical section.
Exit section
Remainder section.
} while (True);

- A critical section is a part of a program where shared resource (like memory, files or variables) are accessed by multiple processes or threads.

° Structure of a critical section.

• Entry Section - The process requests permission to enter the critical section.

- synchronization tools (e.g. mutex, semaphore) are used to control access.

2. Critical section-

Click https://t.me/SPPU_TE_BE_COMP

The actual code where shared resources are accessed or modified.

- Exit section - The process related releases the lock or semaphore, allowing other processes to enter the critical section.
- Remainder section - The rest of the program that does not involve shared resource access.

- In above diagram, the entry section handles the entry into the critical section.
- It acquires the resources needed for execution by the process.
- The exist section handle exist from the critical section.
- It releases the resource & also informs the other processes that the critical section is free.
- The critical section in OS is used to solve the race condition occuring due to process synchronization.
- Shared variables are created & initialized before either process starts.
- Addressing the critical section problem is essential to avoid interference among concurrent processes in an operating system (OS)
- Shared resources include memory, global variables, files, database.

Click https://t.me/SPPU_TE_BE_COMP

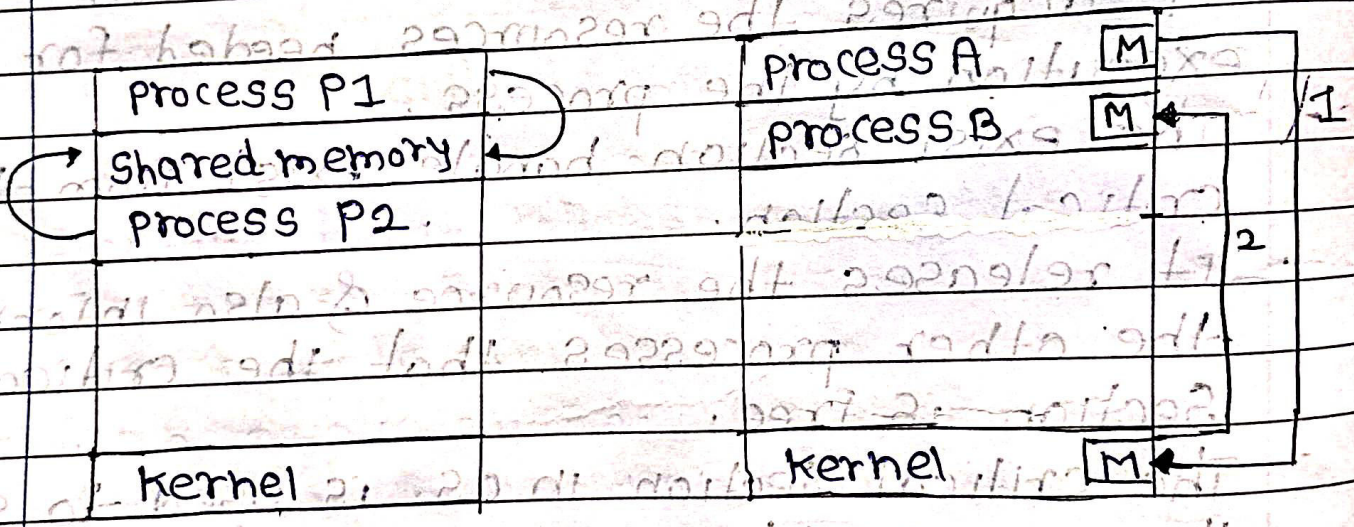
IPC

Interprocess Communication

- What is inter process communication? Why is important for operation system.

→ processes need to communicate with each other in many situations.

- Inter process communication or IPC is a mechanism that allows process to communic
- Interprocess communication is very importan to the design process for microkernels & nanokernels, which reduce the number of functionalities provided by the kernel.



Shared memory Message passing

fig. shared memory & message passing

- There are two fundamental models of inter process communication that are commonly used, these are.
- Shared memory
- message passing

Click https://t.me/SPPU_TE_BE_COMP

- Shared memory:-
  Communication between processes using shared memory require processes to share some variable & it completely depends on how the programmer will implement it.
- In the above shared memory model, a common memory space is allocated by the kernel.
- process A writes data into the shared memory region.
- process B can the directly read this data from the same shared memory region.
- Since both processes access the same memory segment, this method is fast but required synchronization mechanisms to avoid when multiple processes read/write simultaneously.

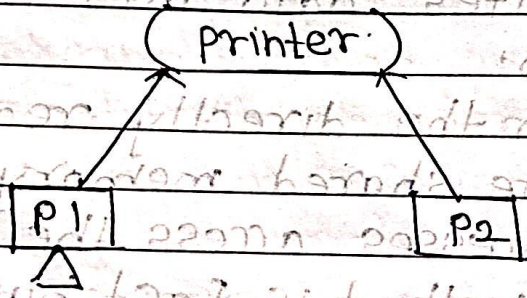
- Message passing.
  Message passing is a method where processes communicates by sending & receiving message to exchange data.
- In this method one process send a message & other process receives it, allowing them to share information.
- Message passing can be achieved through different method like message queue, pipes or sockets.
- So message passing means how a message can be sent from one end to the other end.

Click https://t.me/SPPU_TE_BE_COMP

TOP

Synchronization problem :-

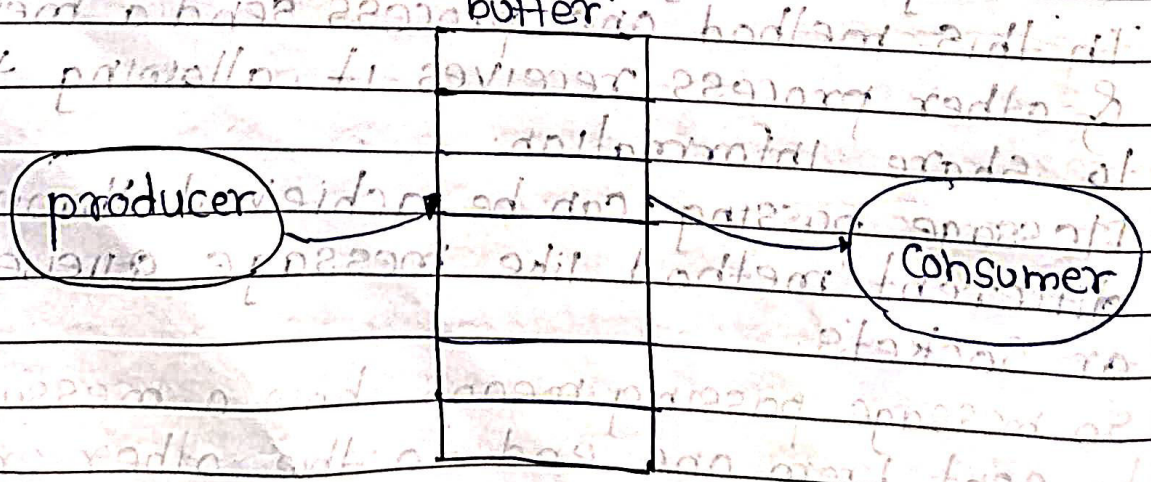
- Synchronization in operating system coordinates concurrent processes to prevent issue like race conditions & data inconsistency when they shared resource.
- Synchronization ensures perfect execution for processes & controlled access to shared resources



There are problems of synchronization are considered as follows

- producer - consumer
- Reader writer
- Dining philosophers.

producer - consumer problem :



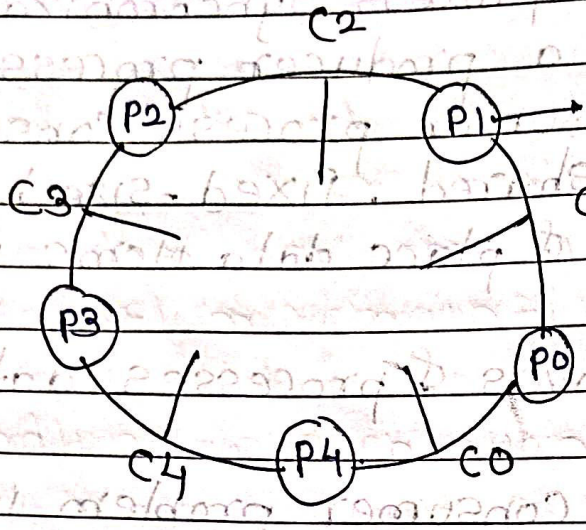
Click https://t.me/SPPU_TE_BE_COMP

- The producer-consumer problem in an OS is a classical multi-process synchronization challenge where a producer process generates data & a consumer process consumes it both using a shared, fixed-size buffer.
- producer - Create & place data items into the buffer.
- Consumer - Removes & processes data items from buffer.
- The producer - Consumer problem is a classic example of synchronization problem in operating system.
- A producer does not add data to a full buffer.
- Multiple producers & consumers do not access the buffer simultaneously preventing race conditions.

- **Reader - Writer:**
- The readers - writers problem relates to an object such as files that is shared between multiple processes.
- Some of these processes are readers ie they only want to read the data from the object & Some of the processes are writers ie they want to write into the object.
- The reader-write problem is a classic synchronization challenge in operating system & concurrent programming.

Click https://t.me/SPPU_TE_BE_COMP

Dining philosophers



- The dining philosopher problem is a classic synchronization & concurrency problem that deals with resource sharin deadlock, where multiple processes required limited resources.
- The life of each philosopher consists prinap of thinking & eating.
- This problem may not seem important or relevant in itself.
- The solution of this problem is giving by semaphores & monitors.

Click https://t.me/SPPU_TE_BE_COMP

Topic | Page No.
Date / /

Step

Explain Bankers Algorithm ?

- The bankers' algorithm is a resource allocation & deadloch avoidance algorithm used in operating system.
- It is designed to manage the allocation of limited resource to processes in a way that prevents deadlock situations.
- The algorithm works by keeping track of the avaliable resources & the maximum resource requirements of each processes. The banker's algorithm operates in a series of steps.

1) **Initilization**: At the beginning, the algorithm initializes data structures & variables to represent the avaliable resources & the maximum resource requirements of each process.

2) **Request Evaluation**: when a process requests addational resources, the algorithms checks if grant the request would leave the system in a safe state.
- It examines wheather the request resources can be satisfied based on the avaliable resources & the maximum requirements of other processes.

3) **Resource Allocation**: If the requested resources can be allocated without causing an unsafe state, the algorithm grants the resources to the requesting process.

Click https://t.me/SPPU_TE_BE_COMP

- It updates the avaliable resource & adjusts the allocation records accordingly.

4) **Safty check:-**
After granting resource, the algorithm performs a safety check to ensure the system remains in a safe state.

5) **Deadlock prevention:-**
It is a requested allocation would result in an unsafe state, the algorithm request to prevent potential deadlocks.
- The process making the request may need to wait until sufficient resource become avaliable.

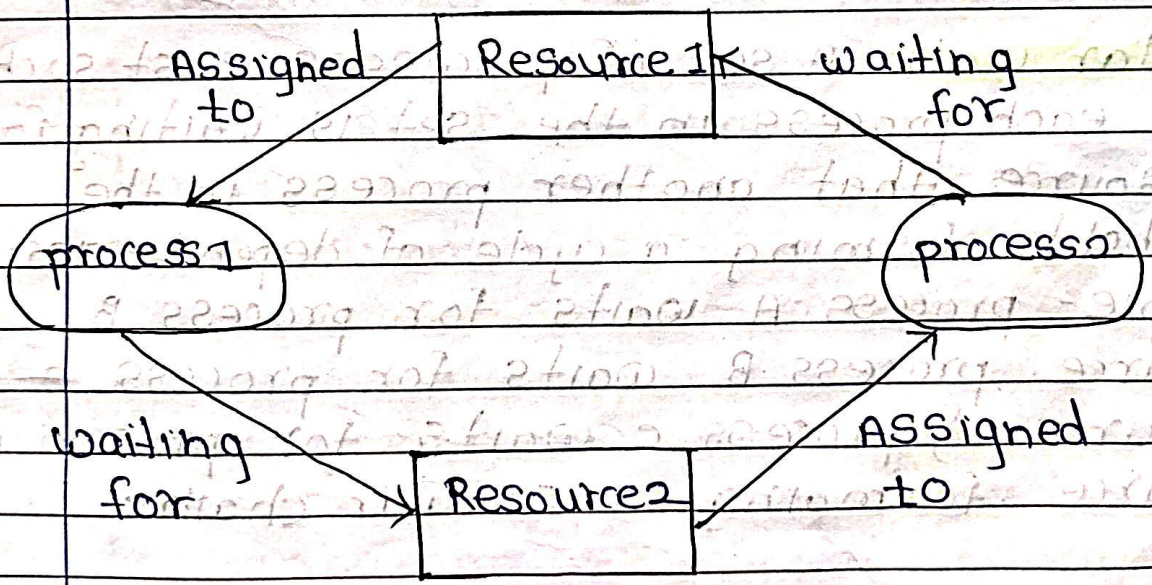
By employing the algorithm, operating system can effectively manages resources & maintain system stability & reliability in multi-process environments.

Ans: **Deadlock:-**
- A deadlock is a situation where a set of processes are blocked because each process is holding a resource & waiting for another resource acquired by some other processes.
- Consider an example when two trains are coming toward each other on the same track & there is only one track, none of

Click https://t.me/SPPU_TE_BE_COMP

the trains can move once they are in front of each other.

- A similar example occurs in operating systems where two or more processes that hold some resources & wait for resource held by other.
- In the below diagram, process 1 is holding Resource 1 & waiting for resource 2 which is acquired by process 2, & process 2 is waiting for resource 1.



- **Condition of Deadlock**
- For deadlock to occur, four necessary condition must hold simultaneously.

- **Mutual Exclusion** - At least one resource must be held in a non-shareable mode, meaning only one process can use it at a time.

Click https://t.me/SPPU_TE_BE_COMP

- Example - A printer can only be used by one process at a time.

- **Hold and Wait** - A process holding at least one resource is waiting to acquire additional resource that are concurrently being held by other processes.
  - Example - process A holds a printer & waiting for a scanner, while process B hold a scanner & waits for the printer.

- **Circular wait** - A set of processes exist such that each process in the set is waiting for a resource that another process in the set holds, forming a cycle of dependencies.
  - Example - process A waits for process B resource, process B waits for process C resource, & process C waits for process A resource, formaing a circular chain.

- **No preemption** - Resources cannot be forcibly taken from processes holding them.
  - A process must release the resource when it is no longer needed.
  - Example - If process A is holding a printer & is waiting for a scanner, the system cannot force process A to given up the printer.

Click https://t.me/SPPU_TE_BE_COMP

Date / /

° Write short note on critical section/monitors & mutex.

1) Critical section-

In concurrent programming, a critical section refers to a part of the program where shared resource are accessed or modified by multiple processes or threads.

- It is important to ensure that only one process or thread can access the critical section at a time to maintain data & prevent race conditions.
- proper synchronization mechanisms, such as monitors, mutexes & used to protect critical section & ensure access.

2) Monitors-

A monitor is a synchronization construct that provides a high level abstraction for managing access to shared resources within concurrent program.

- It combines mutual exclusion & condition variables.
- Monitors encapsulate the shared data & the producers that operates on that data providing a structure & safe way to coordinate access & synchronization.

3) Mutex(Mutual Exclusion)-

Click https://t.me/SPPU_TE_BE_COMP

- A mutex is a binary synchronization primitive used to protect critical sections & ensure access to shared resources.
- A mutex has two states: locked & unlocked.
- only one process or thread can hold the lock of the mutex at a given time.
- once a process or thread finishes its work in the critical section, it releases the lock of the mutex, allowing other processes or threads to acquire it & enter the critical section.

• Explain monitor in brief?

- Monitor is an object that contains both data & procedures needed to perform allocation of a shared resource.
- It is a programming language construct that support both data access synchronization & control synchronization.

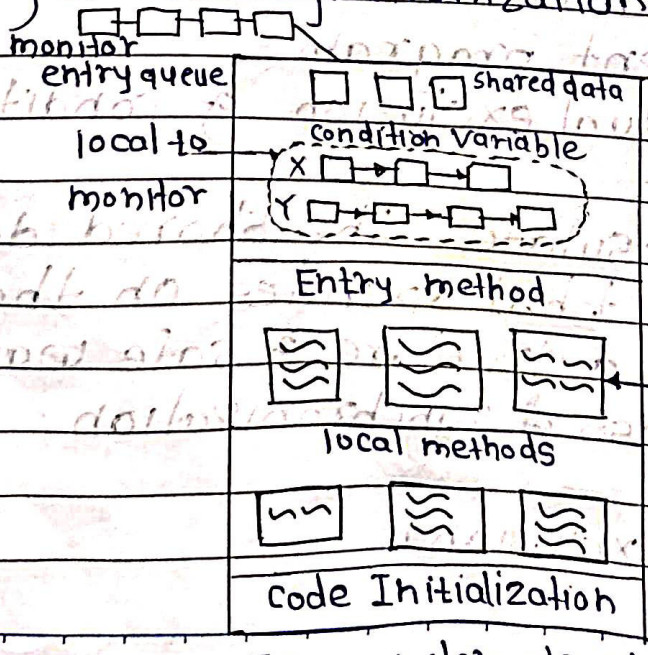


Fig. monitor structure

Click https://t.me/SPPU_TE_BE_COMPS

- monitor is implement in programming language like java, C++.
- Java makes exclusive use of monitor to implement mutex exclusion.
- monitor is an abstract data type for which only process may executing producers at given time. monitor is collection of producers variable & data structure.
- Monitor data is accessible only within the monitor.
- A shared data structure can protected by placing it in monitor.
- If the data represents some resource then monitor provide mutual exclusion facility for accessing the resource.
- Condition variable - It suspend execution of calling process on condition.

Click https://t.me/SPPU_TE_BE_COMP

Assignment No-3

1) What is interprocess communication ? Why is important for operating system.
2) Explain bankers Algorithm ?
3) Explain monitor in berief ?
4) What is the different types of classical synchronization problem ? Explain one in details
5) Write short notes on critical section, monitors & mutex.
6) What is deadlock ? Discuss the necessary condition for deadlock with example.
7) What is critical section problem ? Explain in details ?
8) What is concurrency ? Explain in details ?